

DRC Job Intelligence Engine

End-to-End Technical Architecture, Execution Flows & Production Roadmap

Document Version: 2.0.0 | Date: October 01, 2026 | Target: Engineering Leadership & Management

1. Executive Summary & Big Picture Architecture

The DRC Job Assistant platform delivers sub-20ms job discovery, deterministic ATS resume compatibility scoring, and duplicate-proof multi-batch candidate matching. To understand the architecture simply, consider the **Restaurant Analogy**:

Component	Role in Real World	Technical Stack & Port	Core Responsibility
Frontend (React)	The Waiter	React 18 + Vite (Port 5173)	Captures user requirements (role, city, resume) and renders interactive job cards with ATS badges.
Engine (Python)	The Kitchen / Chef	FastAPI Microservice (Port 5055)	Concurrently scans LinkedIn & ATS portals, calculates deterministic ATS match %, and batches results.
Database Layer	The Pantry / Ledger	SQLite WAL / MySQL Relational	Maintains historical discovery batches, canonical fingerprints, and application tracking states.

2. Complete Codebase File Directory Map

Every file in the codebase has a single, modular responsibility without dead weight:

File Path	Layer	Modular Responsibility
src/App.tsx	Frontend	Central router and screen switchboard (Home, QA-to-BA, Job Discovery, Tracker, Pricing).
src/components/JobDiscovery.tsx	Frontend	Interactive job search UI with instant filters, experience selectors, and batch discovery triggers.
src/components/DiscoveredJobsTracker.tsx	Frontend	Renders structured job cards, ATS fit badges, matched skill pills, and live application links.
src/lib/drcDiscoveryApi.ts	Frontend API	The Bridge: TypeScript client making typed REST calls to the FastAPI Job Engine.
job_engine/api_server.py	Backend Core	Main Microservice: FastAPI server hosting 12 REST endpoints for search, scoring, and DB ops.
job_engine/two_source_matcher.py	Backend Engine	Parallel orchestrator executing concurrent LinkedIn and Career Page queries with cross-source deduplication.
job_engine/job_matcher.py	Backend Engine	Real-time LinkedIn Guest API scraper extracting verified job IDs, locations, and live URLs.
job_engine/career_pages_engine.py	Backend Engine	Direct enterprise ATS crawler indexing open requisitions across Workday, Greenhouse, Lever, and Ashby.

job_engine/ats_calculator.py	Backend ML/Alg	Multi-dimensional ATS scorer: 45% Skill Match + 35% Title/Domain Match + 20% Experience Curve.
job_engine/deterministic_parser.py	Backend Parser	High-speed (<100ms) Zero-LLM resume text extractor for contact info, skills, and seniority.
job_engine/db_store.py	Backend DB	Thread-safe SQLite/WAL repository managing deduplication fingerprints, batches, and candidate status.
job_engine/query_expander.py	Backend Taxonomy	Semantic technology taxonomy mapping queries (e.g., MERN -> React, Node, Express, Mongo).
job_engine/india_locations.py	Backend Geo	Pan-India canonical location resolver mapping metro clusters and remote configurations.

3. Step-by-Step Search & Discovery Execution Journey

When a candidate clicks 'Find Jobs' in the UI, the system executes a 9-step sub-second pipeline:

- Step 1: UI Trigger (React):** Candidate selects domain/location (e.g. 'React Developer in Bengaluru') and clicks Find Jobs.
- Step 2: Bridge Client (drcDiscoveryApi.ts):** Constructs typed CandidateSearchPayload and dispatches POST /api/jobs/search.
- Step 3: Vite Dev Proxy / Nginx Gateway:** Forwards request from port 5173 to local microservice on port 5055 with sub-millisecond latency.
- Step 4: FastAPI Router (api_server.py):** Validates payload via Pydantic model and invokes execute_fresh_batch_discovery().
- Step 5: Parallel Dual-Source Ingestion:** Spawns concurrent worker threads querying LinkedIn Guest API (Source 1) and Direct ATS Portals (Source 2).
- Step 6: Semantic Expansion & India Routing:** query_expander.py injects synonym clusters; india_locations.py resolves regional metro clusters.
- Step 7: ATS Compatibility Matrix Evaluation:** ats_calculator.py compares candidate resume tokens vs JD requirements (45/35/20 split).
- Step 8: Three-Layer Duplicate Guard & Persistence:** db_store.py checks LinkedIn ID, Canonical URL, and SHA-256 fingerprint; saves batch to DB.
- Step 9: Real-Time UI Hydration:** React renders sorted job cards with ATS score badges, matched skill pills, and live application URLs.

4. The Bridge: React to Python Communication

The React frontend communicates with the Python engine via clean REST contracts. In development, Vite acts as a reverse proxy, eliminating CORS barriers:

Vite Proxy Configuration (vite.config.ts):

```
// vite.config.ts
server: {
  port: 5173,
  proxy: {
    '/api': {
      target: 'http://127.0.0.1:5055', // Python FastAPI Microservice
      changeOrigin: true,
      secure: false
    }
  }
}
```

Frontend API Client Methods (src/lib/drcDiscoveryApi.ts):

Function	HTTP Route	Payload / Return Type
searchInstantJobs()	POST /api/jobs/search	Instant sub-20ms 5-fresh-job allocation with duplicate guard.
findMatchingJobs()	POST /api/find-jobs	Personalized matching against candidate's parsed resume profile.
findTwoSourceJobs()	POST /api/find-two-source-jobs	Side-by-side comparative LinkedIn and Career Page listings.
getDiscoveredJobsHistory()	GET /api/discovered-jobs	Loads all historical discovery batches for candidate email.
updateJobStatus()	POST /api/update-job-status	Persists Applied / Not Applied status and custom remarks.
checkJobEngineHealth()	GET /api/health	Health probe returning service status and engine version.

5. Two-Source Job Discovery Engine

To maximize candidate conversion and eliminate expired postings, the engine ingests from two distinct live sources concurrently:

Source 1: Real-Time LinkedIn Guest API (job_matcher.py)

- Queries LinkedIn's public guest search endpoints without requiring user login or credentials.
- Applies semantic role expansion and geo-clustering across India Metro hubs (Bangalore, Pune, Hyderabad, Gurgaon, Mumbai).
- Extracts verified numeric Job IDs (e.g. 4102938476) and builds direct view URLs: <https://www.linkedin.com/jobs/view/{id}>.

Source 2: Direct Enterprise ATS & Career Pages (career_pages_engine.py)

- Directly indexes company career portals powered by Workday, Greenhouse, Lever, Ashby, and SmartRecruiters.
- Scrapes open corporate requisitions with guaranteed 100% active application links.
- Eliminates recruiter middlemen and gives candidates direct access to hiring managers.

6. ATS Compatibility Scoring Algorithm

The scoring algorithm in `ats_calculator.py` computes a deterministic compatibility score (0–100%) across three balanced dimensions:

Evaluation Dimension	Weight	Scoring Formula & Logic
1. Skill Matrix Match	45% Max	Compares candidate's parsed skills against JD key skills and full text. Matches are added to <code>matched_skills</code> , gaps to <code>missing_skills</code> . Score = (Matched / Required) * 45.
2. Title & Domain Fit	35% Max	Evaluates token overlap between candidate's primary domain / target roles and the job title. Domain hit = +12 pts; Role hit = +8 pts (clamped between 10 and 35).
3. Experience Curve	20% Max	Compares candidate years vs JD range (e.g. 3-5 yrs). Exact fit = 20 pts; 1 yr gap = 16 pts; 2.5 yr gap = 11 pts; Overqualified/Senior fit = 17-19 pts.
Total ATS Score	100%	Tiers: Excellent (85–100%) Good (70–84%) Fair (55–69%) Low (<55%)

7. Database Architecture & Three-Layer Duplicate Guard

The database repository (`db_store.py`) operates in SQLite Write-Ahead Logging (WAL) mode for concurrency and zero lock contention:

Primary Relational Tables:

- `discovered_jobs`: Master ledger of every discovered requisition with canonical URL, fingerprint, and ATS metrics.
- `discovery_batches`: Sequential batch records (Batch 1, Batch 2...) tracking candidate discovery sessions.
- `candidates`: Candidate profiles, parsed skills, domain preferences, and resume texts.
- `candidate_applications`: Submitted job applications for recruiter tracking and follow-up.
- `companies` & `job_skills`: Normalized company metadata and 1-to-many indexed technical skills.

Three-Layer Duplicate Guard Logic:

1. **LinkedIn Job ID Layer**: Extracts numeric ID from URL. If ID was previously discovered for candidate -> *Discard*.
2. **Canonical URL Layer**: Strips query parameters, UTM tags, and session trackers to base URL -> *Discard if seen*.
3. **Company + Title Fingerprint Layer**: Normalizes company (removes 'Pvt Ltd', 'Tech') and title (removes 'Senior', 'Lead') into SHA-256 fingerprint -> *Prevents duplicate listings under slightly different titles*.

8. Production Integration: Java + MySQL + AWS EC2

In your production environment, the Python Job Engine functions as a **dedicated internal microservice** alongside your Java Spring Boot backend and MySQL database on AWS EC2:

Production Deployment Architecture:

- **Nginx (Port 443 / SSL)**: Serves compiled React static assets (`/var/www/html`) and reverse proxies `/api/*` to Java Spring Boot.
- **Java Spring Boot (Port 8080)**: Handles authentication, user management, payments, and connects to AWS RDS MySQL via HikariCP.
- **Python Job Engine (Port 5055)**: Companion microservice bound to `127.0.0.1` (internal only). Scrapes, parses, and scores jobs upon Java request.

- **MySQL Database (AWS RDS / Port 3306):** Persists candidates, job matches, and application statuses across all microservices.

Sample Java Spring Boot Integration Service (JobDiscoveryService.java):

```
@Service
public class JobDiscoveryService {
    private final RestTemplate restTemplate = new RestTemplate();
    private static final String PYTHON_ENGINE = "http://127.0.0.1:5055";

    public List<JobDTO> discoverJobs(CandidateDTO candidate, String role, String location) {
        Map<String, Object> request = Map.of(
            "query", role,
            "location", location,
            "candidate", Map.of(
                "name", candidate.getName(),
                "email", candidate.getEmail(),
                "top_skills", candidate.getSkills(),
                "total_experience_years", candidate.getExperience()
            ),
            "limit", 5,
            "source", "all"
        );

        // Call Python Microservice (exact same JSON contract as React)
        ResponseEntity<Map> response = restTemplate.postForEntity(
            PYTHON_ENGINE + "/api/jobs/search", request, Map.class
        );

        List<Map> jobs = (List<Map>) response.getBody().get("jobs");
        return jobs.stream().map(this::saveAndConvertToDTO).collect(Collectors.toList());
    }
}
```

Key Takeaway for Management: The Python Job Engine is a self-contained, microservice-ready system. It decouples complex web scraping and ATS scoring algorithms from the main application. In production on AWS EC2, Java Spring Boot simply invokes the Python engine's existing REST endpoints internally with **zero code changes required** on the engine layer.